JC2001: Introduction to Software Engineering

Group Project Proposal

PlayPair

An AI-Assisted Decision Agent for Planning Group Outings

Group number: CS10
Programme of study: BSc (CS)
Supervisor(s): Jovan, Ferdian
Submission date: 25 September 2026

Group Members

No.	Student name	Student ID
1	Jincheng Huang	50106121
2	Zixuan Liang	50106098
3	Ziqing Liu	50106104
4	Jingyi Hu	50106088
5	Baian Chen	50106103
6	Jinghao Zhang	50106027
7	Ruixuan Qiu	50106105
8	Jiayi Zhong	50106141
9	Hongyi Xie	50106101
10	Zhihao Chen	50106120

South China Normal University

Academic Year 2026–27

Title

PlayPair: An AI-Assisted Decision Agent for Planning Group Outings

Problem

Planning an outing among friends requires a compatible time, an acceptable activity and place, and respect for practical limits such as budget or accessibility. Availability and preferences are usually scattered across messages and informal polls. This makes it easy to miss a hard constraint or continue discussing options without settling on a plan. Calendar, polling and venue-listing tools address parts of the task, but they do not combine these inputs into a transparent group recommendation. PlayPair targets existing friendship groups of young people in the same city. Its proof of concept focuses on outings near South China Normal University (SCNU). Interviews and a short questionnaire will test which coordination difficulties matter most before the requirements are finalised.

Solution

PlayPair is a web-based decision agent that carries a group from invitation to a confirmed outing. In Alpha, users register and log in; an initiator creates a group and other members join with its code. The initiator creates an outing and invites group members. Invitees accept or decline, and participating members enter availability, hard constraints and soft preferences through the web interface. The decision agent works behind the scenes rather than conversing with users. The system presents up to three complete time–activity–place options in ranked order. The initiator chooses and confirms one option, which then appears in *My Outings*. Invitation response and later attendance are distinct recorded states. Alpha records changes in attendance but does not automatically re-plan after a decline or absence.

1. **Exact constraints and candidate construction.** The Go back end combines participating members' available times with a fixed, verified catalogue of approximately 20–40 activities and venues near SCNU. It constructs candidates and removes any that breach declared hard constraints, including budget, accessibility and explicit exclusions. It does not calculate each member's travel distance. The system never silently relaxes a hard constraint; if fewer than three feasible options exist, it displays those available, and if none exist, it asks users to revise their inputs.
2. **Bounded AI scoring.** After filtering, a structured *DecisionProvider* scores every feasible candidate for its fit with participating members' soft preferences. Jev is the intended provider.[1] An ordinary LLM provider uses the same scoring schema if Jev is unavailable; a Mock provider supports demonstrations without an API key and is not presented as AI

judgement. Go constructs the candidates and applies hard constraints; no provider creates an unverified venue or overrides that result. Alpha uses the scores to rank feasible candidates and shows up to three options. Exact weights and thresholds will be informed by later experiments. Alpha does not expose a reason for the model's score or promise a preference-level explanation; its user-facing guarantee is that every displayed option satisfies the participating members' declared hard constraints.

3. **Decision records and required Beta recommendation text.** The system records the inputs, catalogue version, constraint results, provider scores and final ranking. In Beta, a separate LLM API is a required part of the recommendation path: it turns this recorded decision evidence into a concise recommendation for each displayed option. The text can describe supported preference matches and trade-offs between options, and may name a member when that member's soft preference is sufficiently important to the recommendation and the agreed data-sharing rules permit it. The LLM decides whether naming a member adds useful context; the application supplies the relevant recorded preferences and does not require every recommendation to name someone. The text must not invent constraints, venues, member preferences or reasons for a provider's internal score. The records make the recommendation process traceable and support checks that the generated text is faithful to the inputs; they do not explain the scoring model's internal reasoning. If this API fails, the recommendation service reports a service failure rather than silently treating the text as an optional enhancement. The initiator retains the final decision in Alpha; no member vote or confidence-triggered vote is required.

The catalogue is deliberately bounded: Alpha has no map or Places API, no personal-distance calculation, and no LLM web search or free venue recommendation. Beta adds required LLM-generated recommendation text, member voting with stored votes and result handling, and separates confirmed outings in *My Outings* into upcoming and historical entries according to the outing date. The LLM used to write this text has a different role from a provider that scores candidates. The exact voting completion rule can be settled during Beta design. Conversational follow-up on a recommendation is outside Alpha and Beta. Later experiments may compare rules-only, Jev and LLM scoring and investigate ranking policy and explanation quality; these comparisons are not prerequisites for Alpha.

Objectives

- **Objective 1: Validate the problem and specify requirements.** By 12 October 2026, study the target users' coordination needs, define the core use cases and measurable requirements, and review scope and research ethics with the supervisor.
- **Objective 2: Deliver the complete Alpha journey.** By 12 October 2026, demonstrate accounts, group-code joining, invitations and responses, member inputs, feasible candidates

from the SCNU catalogue, provider scoring, top-three presentation, initiator confirmation and *My Outings*. Every displayed proposal must satisfy the participating members' declared hard constraints.

- **Objective 3: Add Beta recommendation text and voting.** By 2 November 2026, integrate an LLM API to write a grounded recommendation for each ranked option from recorded evidence; add Beta voting and the upcoming/historical *My Outings* views.
- **Objective 4: Evaluate and submit the system.** By 23 November 2026, test the workflow, hard-constraint compliance, provider failures and usability; then complete the runnable proof of concept, user manual, report and presentation by 14 December 2026.

Benefits

- **Less coordination effort.** One shared process replaces scattered messages for invitations, availability, constraints and final confirmation.
- **Practical options and user control.** The verified local catalogue and exact filtering make proposals reproducible and prevent a model from inventing places or ignoring declared limits. The initiator decides whether a proposal becomes an outing.
- **A clearer choice in Beta.** Required, concise, evidence-grounded recommendation text helps the group compare preference matches and trade-offs instead of receiving only a ranked list. Its faithfulness to recorded inputs will be evaluated, including whether optional member references are supported by the evidence.
- **Useful engineering evidence.** The replaceable provider and decision record permit repeatable comparisons of scoring approaches and failure handling without tying the core workflow to Jev availability.

These benefits are hypotheses to evaluate with interviews, task-based usability sessions and recorded system tests, subject to the supervisor's guidance on research ethics.

Timeline

Phase	Main outcome	End date
1	Confirm scope with the supervisor, submit the proposal, prepare user research and request Jev access.	25 Sep 2026

Phase	Main outcome	End date
2	Complete requirements and Alpha: accounts, group joining, invitations, member inputs, catalogue, filtering, interchangeable scoring, decision records, confirmation and *My Outings*.	12 Oct 2026
3	Integrate the required LLM recommendation-text API, handle its service failures, check text against decision records, and add Beta voting and date-based outing views.	2 Nov 2026
4	Freeze scope; test the integrated system, run decision and usability evaluations, and collect report evidence.	23 Nov 2026
5	Fix critical defects and prepare the runnable package, user manual, report draft and presentation.	7 Dec 2026
6	Recheck reproducibility and consistency; submit the report, proof of concept, manual, slides and recording.	14 Dec 2026

The phase boundaries coincide with the scheduled project updates on 12 October, 2 November, 23 November and 7 December 2026.

Action Plan

The assignments below retain the team's v2 allocation. Progress is a planning snapshot for 25 September 2026 and should be updated by the group as work proceeds.

Action	Assigned to	Deadline	Status
Objective 1: Requirements			
Discuss scope, research ethics and success measures with supervisor.	Jincheng Huang, Ruixuan Qiu	25 Sep 2026	In progress
Conduct 6–10 interviews and a short questionnaire, subject to ethics guidance.	Jingyi Hu, Hongyi Xie, Jiayi Zhong	8 Oct 2026	Not started
Define use cases, requirements and traceability to planned tests.	Jingyi Hu, Hongyi Xie, Jiayi Zhong	12 Oct 2026	Not started
Objective 2: Alpha			

Action	Assigned to	Deadline	Status
Design account, group, invitation, outing, catalogue and decision data contracts.	Jincheng Huang, Baian Chen, Hongyi Xie	30 Sep 2026	Not started
Build and verify the fixed SCNU catalogue, including source and last-check details.	Baian Chen, Jiayi Zhong	3 Oct 2026	Not started
Implement and test Go candidate construction and hard-constraint filtering.	Jincheng Huang, Baian Chen	7 Oct 2026	Not started
Implement the Jev, LLM and Mock scoring providers behind one schema; store inputs, constraint outcomes and scores.	Jincheng Huang, Baian Chen, Ruixuan Qiu	10 Oct 2026	Not started
Deliver accounts, group-code joining, invitation responses, member inputs, scoring, top three, initiator confirmation and *My Outings*.	Jingyi Hu, Zhihao Chen, Zixuan Liang, Ruixuan Qiu, Ziqing Liu	12 Oct 2026	Not started
Objective 3: Recommendation text and Beta			
Define the LLM recommendation-text input, permitted evidence, optional member references and review checks; review scoring-provider fallback behaviour.	Jincheng Huang, Baian Chen, Ruixuan Qiu	30 Oct 2026	Not started
Integrate the required LLM API for grounded recommendation text, including service-failure handling; add member voting and upcoming/historical outing views.	Jincheng Huang, Baian Chen, Jingyi Hu, Zhihao Chen	2 Nov 2026	Not started
Objective 4: Evaluation and submission			

Action	Assigned to	Deadline	Status
Run unit, interface, end-to-end and failure-path tests, including hard-constraint and provider-failure cases.	Baian Chen, Jinghao Zhang	16 Nov 2026	Not started
Compare feasible-candidate rankings with a rules-only baseline and assess acceptance, latency, cost and explanation fidelity.	Jincheng Huang, Baian Chen	20 Nov 2026	Not started
Conduct task-based usability evaluation and document limitations.	Jincheng Huang, Baian Chen, Ruixuan Qiu	23 Nov 2026	Not started
Complete report, models, references and group workload profile.	All members	7 Dec 2026	Not started
Complete the 8–12 page manual and verify clean installation of the runnable package.	All members	7 Dec 2026	Not started
Rehearse the 15-minute presentation and check all final submissions.	All members	14 Dec 2026	Not started

References

[1] TypeSafe AI, “Introduction” and “How to build with TypeSafe”, *TypeSafe AI Documentation*, 2026. Available: docs.typesafe.ai. Accessed: 22 September 2026.